Java Process Table Function (PTF) User-Defined Function (UDF) type — Session Timeout Detector, a timer-driven PTF example

> The Session Timeout Detector is driven by **event-time timers** that fire when a user becomes inactive.
>
> Unlike the companion User Event Enricher (which is purely row-driven — state transitions triggered only by incoming rows), this PTF uses Flink's timer service to schedule future actions and react when those timers fire.
>
> This example demonstrates how the Process Table Function (PTF) API in Flink 2.1+ enables building fully stateful, **timer-driven** operators in Java that are directly callable from SQL.

Table of Contents

- **1.0 Timers and timer-driven processing in a Process Table Function**
  - **1.1 What are timers?**
  - **1.2 How timers work in a PTF**
  - **1.3 Named timers and the inactivity pattern**
  - **1.4 The role of `on_time`, `TimeContext`, and `OnTimerContext`**
- **2.0 What does this example do?**
  - **2.1 Detection logic**
  - **2.2 How it works end-to-end**
  - **2.3 Key concepts illustrated**
- **3.0 Row-driven vs timer-driven — comparing the two PTF examples**
- **4.0 Resources**

1.0 Timers and timer-driven processing in a Process Table Function

A **Process Table Function (PTF)** is a stateful operator that sits inside the Flink dataflow graph. The companion example showed how a PTF can maintain per-user state and transition that state purely in response to incoming rows. This example adds a second dimension: **time**.

1.1 What are timers?

In stream processing, **timers** let an operator schedule a callback at a future point in time. Without timers, an operator can only react when a new row arrives. With timers, it can also react when *nothing happens* — detecting inactivity, enforcing deadlines, or triggering periodic side-effects.

Flink manages timers as a special kind of state:

- Timers are persisted in the **state backend** alongside your other state
- They survive **checkpoints and restores** — if the job restarts, pending timers are restored and fire at the correct time
- They are scoped to a **partition key** — each `PARTITION BY` key has its own independent set of timers

1.2 How timers work in a PTF

In a PTF, timers are accessed through the `TimeContext` obtained from the `Context` parameter:

```
TimeContext<Instant> timeCtx = ctx.timeContext(Instant.class);

// Register a named timer that fires 5 minutes from now
timeCtx.registerOnTime("inactivity",
timeCtx.time().plus(Duration.ofMinutes(5)));
```

When the timer fires, Flink calls the `onTimer()` method on the same PTF instance with the same partition key's state:

```
public void onTimer(OnTimerContext onTimerCtx, SessionState state) {
    // Timer fired — the user has been inactive for 5 minutes
    collect(Row.of("session_timeout", state.lastPayload, state.eventCount,
true));
}
```

1.3 Named timers and the inactivity pattern

Timers can be **named** or **unnamed**:

Type	Registration	Behaviour on re-register
Named	`registerOnTime("name", time)`	**Replaces** the existing timer with the same name
Unnamed	`registerOnTime(time)`	**Adds** a new timer (previous ones still fire)

Named timers are essential for the **inactivity pattern**: on every event, re-register a timer with the same name. Each new event replaces the previous timer, effectively resetting the inactivity clock. If no new event arrives, the timer fires.

1.4 The role of `on_time`, `TimeContext`, and `OnTimerContext`

Three pieces connect timers to the PTF:

- **`on_time => DESCRIPTOR(event_time)`** — the SQL-side declaration that tells Flink which column carries the event timestamp. This is required for the PTF to access `TimeContext`.
- **`@ArgumentHint(ArgumentTrait.REQUIRE_ON_TIME)`** — the Java-side annotation that declares the PTF requires an `on_time` argument. Without this, the `TimeContext` is not available.
- **`TimeContext<Instant>`** — provides `time()` (current event timestamp), `registerOnTime()`, `clearTimer()`, and `currentWatermark()`.
- **`OnTimerContext`** — passed to `onTimer()`, identifies which timer fired via `currentTimer()`.

2.0 What does this example do?

This example puts the above concepts into practice. The `SessionTimeoutDetector` PTF reads user activity events from a Kafka topic (`user_activity`), monitors per-user inactivity, and writes enriched output (including timeout alerts) to a second Kafka topic (`timeout_events`).

2.1 Detection logic

For every incoming event the function maintains three pieces of **per-user state** (one state instance per `PARTITION BY user_id` key):

State field	Purpose
`eventCount`	Running count of events received in the current session. Reset to zero when the timer fires (session timeout).
`lastEventType`	The `event_type` of the most recent event. Used in the timeout row for context.
`lastPayload`	The `payload` of the most recent event. Carried forward in the timeout row.

The output schema is:

```
user_id      STRING    — passed through automatically via PARTITION BY
event_type   STRING    — original type, or "session_timeout" when timer
fires
payload      STRING    — original payload, or last-seen payload on timeout
event_count  BIGINT    — total events seen before this output
timed_out    BOOLEAN   — false for regular events, true for timeout events
```

2.2 How it works end-to-end

```
Kafka (user_activity)
        │
        ▼
  ┌──────────────┐
  │ user_activity │   Flink SQL source table (JSON / event-time watermark)
  └──────────────┘
        │
        ▼
  SessionTimeoutDetector(
      input   => TABLE user_activity PARTITION BY user_id,
      on_time => DESCRIPTOR(event_time)
  )
        │
        │  Per-user timer-driven detection:
        │    • every event → update state, reset 5-min timer, emit
enriched row
        │    • timer fires → emit "session_timeout" row, clear state
        │
        ▼
  ┌──────────────┐
```

```
    │ timeout_events │    Flink SQL sink table → Kafka (timeout_events)
    └────────────────┘
```

Example timeline for user `alice`:

Time	Event	What happens	Output
T+0s	`login`	State: count=1. Timer set for T+5m.	`(login, web, 1, false)`
T+30s	`click`	State: count=2. Timer reset to T+5m30s.	`(click, button-home, 2, false)`
T+2m	`purchase`	State: count=3. Timer reset to T+7m.	`(purchase, order-42, 3, false)`
T+7m	*(timer fires)*	No event since T+2m.	`(session_timeout, order-42, 3, true)`

2.3 Key concepts illustrated

- **`ProcessTableFunction`** — the Flink 2.x API for stateful, set-semantic UDFs callable from SQL.
- **`@StateHint`** — declares a POJO whose fields Flink automatically persists per partition key.
- **`@ArgumentHint(ArgumentTrait.SET_SEMANTIC_TABLE, ArgumentTrait.REQUIRE_ON_TIME)`** — declares that the input is a keyed, stateful virtual processor that requires timer support via an `on_time` argument.
- **`TimeContext`** — provides access to the current event timestamp and timer registration/cancellation.
- **`onTimer()`** — the callback that fires when a registered timer elapses.
- **Named timers** — re-registering a timer with the same name replaces it, enabling the inactivity pattern.

3.0 Row-driven vs timer-driven — comparing the two PTF examples

Aspect	**User Event Enricher (row-driven)**	**Session Timeout Detector (timer-driven)**
Trigger	Every incoming row	Every incoming row **+ timer fire**
Uses timers	No	Yes (`TimeContext`, `onTimer()`)
`on_time` argument	Not required	**Required** (`DESCRIPTOR(event_time)`)
`@ArgumentHint` traits	`SET_SEMANTIC_TABLE`	`SET_SEMANTIC_TABLE` + `REQUIRE_ON_TIME`
State transitions	Row-driven only (login → new session)	Row-driven (update count, reset timer) + timer-driven (timeout → clear state)
Output trigger	One output per input row	One output per input row **+ one output per timer fire**

Aspect	User Event Enricher (row-driven)	Session Timeout Detector (timer-driven)
Use case	Enrichment, session tracking	Inactivity detection, deadlines, SLA monitoring

4.0 Resources

- Apache Flink User-defined Functions
- Create a User-Defined Function with Confluent Cloud for Apache Flink
- Process Table Functions in Confluent Cloud for Apache Flink